

Framework SDLC Agéntico

Un estándar reutilizable para que cualquier equipo de desarrollo trabaje con agentes de forma auditable

Qué es

Un ciclo de vida de desarrollo ejecutado por agentes con roles separados, contratos entre ellos y gates verificables por script. Todo vive en un repositorio Git: no hay nada que configurar en una consola, ni servicio que mantener. Un equipo lo adopta clonando y ajustando dos archivos.

Documento compañero

Fundamentos · Trabajar con agentes cubre los fundamentos que sostienen este framework y que sirven aunque no lo adoptes: los tres niveles, la estructura de una instrucción y las técnicas contra la respuesta inventada. Empieza por ahí si es tu primer contacto con el tema.

Por qué se puede reutilizar

Los procedimientos viven en un catálogo compartido; los umbrales viven en cada equipo. Esa separación es lo que permite que cinco equipos con exigencias muy distintas usen exactamente las mismas instrucciones y solo cambien los números.

1 · El problema que resuelve

Un agente sin estructura hace las siete cosas a la vez: interpreta la solicitud, diseña, implementa, se autoevalúa, declara que está seguro y despliega. El resultado se lee bien y nadie puede auditarlo. No porque el agente sea malo, sino porque no hay ningún punto de la cadena donde algo pueda ser rechazado.

Los tres fallos concretos que aparecen sin estructura:

- Autoevaluación. Quien produce el artefacto lo revisa con el mismo razonamiento que ya usó, así que confirma lo que ya creía.
- Alcance que crece solo. Al implementar aparece un hueco de diseño, el agente lo rellena, y lo que se despliega ya no es lo que se aprobó.
- Evidencia declarativa. «Las pruebas pasan» es una afirmación, no una evidencia. Sin un archivo que alguien pueda abrir, no hay nada que auditar.

Este framework responde a los tres con una idea simple: separar la autoridad. Cada rol tiene entrada, salida y una condición verificable para pasar el trabajo al siguiente. Nadie certifica su propio trabajo.

2 · La cadena

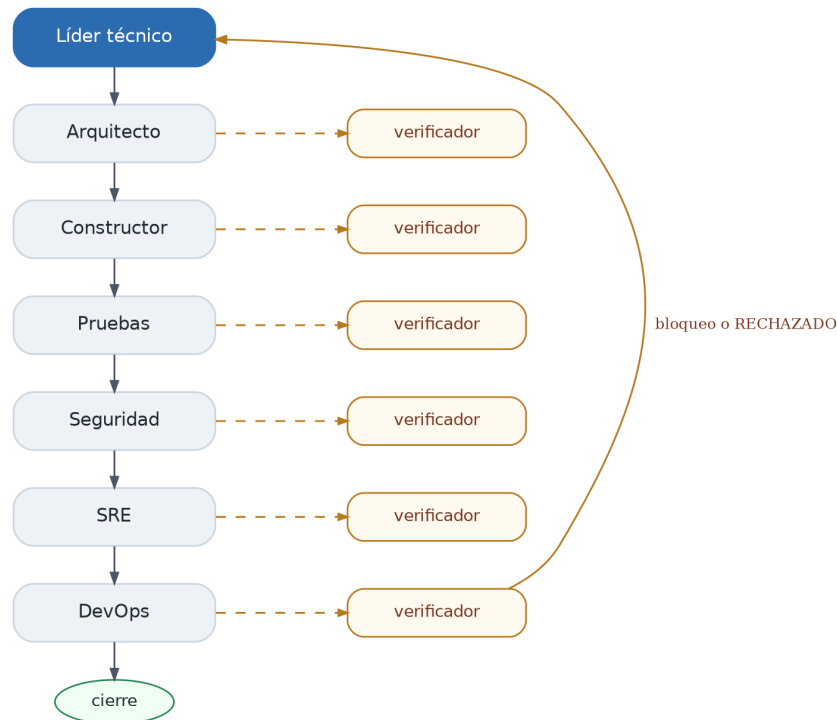


Figura 1 — Cadena de roles. Siete roles secuenciales más un verificador adversarial que se intercala después de cada rol productor. Cualquier bloqueo vuelve al líder técnico, nunca al rol anterior.

Rol	Produce	No hace
Líder técnico	Orden de trabajo con alcance acotado e impacto declarado; cierre	Diseñar, implementar
Arquitecto	Diseño, contratos, matriz de impacto	Implementar
Constructor	Código, pruebas unitarias, inventario de artefactos	Decidir arquitectura, desplegar
Pruebas	Certificación con evidencia de ejecución	Corregir defectos
Seguridad	Dictamen y hallazgos con criterio de cierre	Corregir hallazgos
SRE	Objetivos, alertas, runbook, criterio de reversión	Desplegar
DevOps	Manual de actualización, orden cerrada, ejecución	Diseñar, corregir
Verificador	Veredictos sobre las afirmaciones de cada rol	Corregir el artefacto

Por qué el líder técnico y el arquitecto están separados

Es la separación que más se cuestiona al adoptar el framework. La razón: si el mismo rol asigna trabajo y produce el diseño, no hay quién desempate cuando el diseño resulta ser más grande que la orden. Uno decide qué entra, el otro decide cómo se hace. En equipos pequeños puede ser la misma persona alternando roles — pero no el mismo paso de la cadena.

3 · Las tres capas y por qué se separan

Aquí está la decisión que hace el framework reutilizable. Tres cosas que suelen ir mezcladas en un mismo archivo se separan porque cambian a ritmos distintos y tienen dueños distintos.

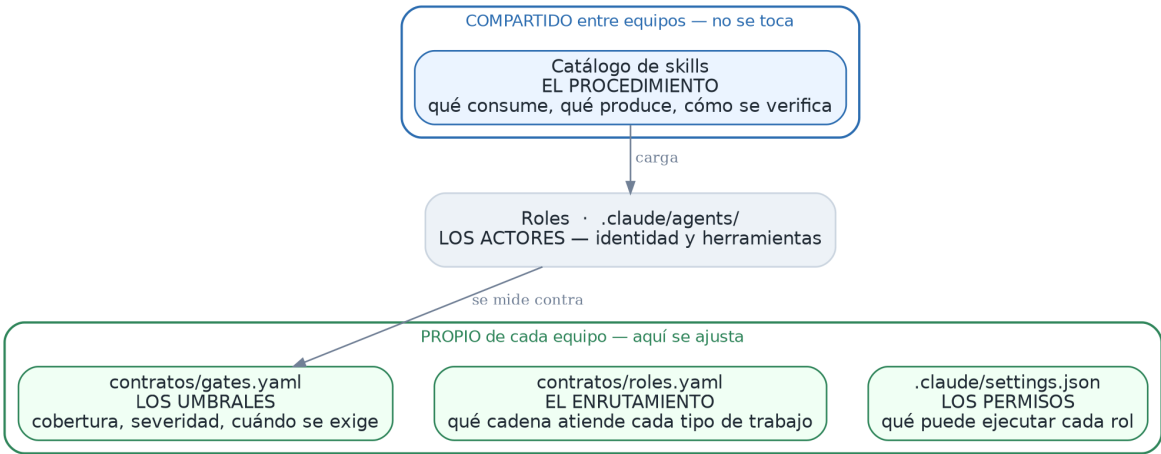


Figura 2 — Separación de capas. El procedimiento es común a todos los equipos; los umbrales, el enrutamiento y los permisos son de cada uno.

Capa	Qué contiene	Ejemplo	Dueño
Rol	Identidad, permisos, herramientas	.claude/agents/sre.md	Plataforma
Skill	El procedimiento: qué consume, qué produce	sre/SKILL.md	Comunidad de práctica
Gate	El umbral que decide si el trabajo pasa	gates.yaml	Cada equipo

La regla que sostiene todo

Ningún número vive dentro de una instrucción. La skill de pruebas dice «la cobertura mínima la fija el equipo en gates.yaml; no la negocies aquí». El número está en un archivo YAML que un script lee.

Esto tiene una consecuencia que va más allá de la limpieza: un agente puede ignorar una instrucción escrita en un prompt — les pasa a todos, en algún porcentaje de las veces. Lo que no puede hacer es que un script de verificación devuelva cero cuando el archivo de evidencia no existe. Mover el control del prompt al dato es mover el control de la persuasión a la aritmética.

4 · La constitución del proyecto

Encima de las tres capas anteriores hay una cuarta que no se ajusta al adoptar el framework: los límites que ningún rol, ninguna orden de trabajo y ninguna presión de calendario pueden cruzar.

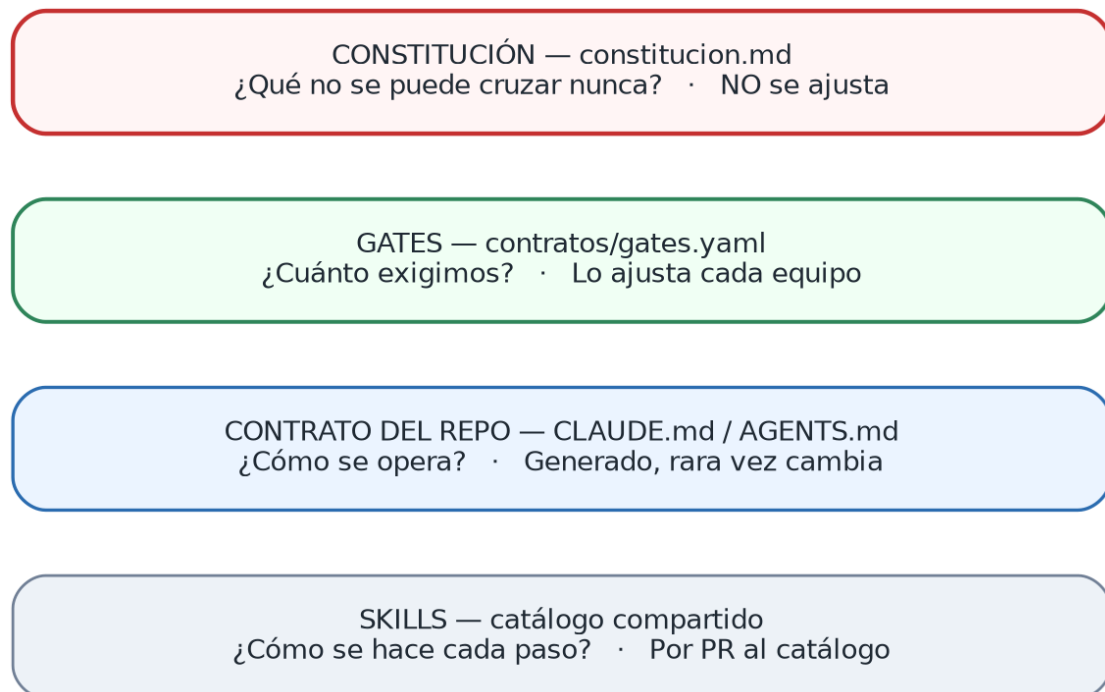


Figura 3 — Las cuatro capas por grado de negociabilidad. Solo la segunda se toca de forma rutinaria.

Los siete artículos

#	Artículo	Lo que prohíbe, en concreto
1	La evidencia manda sobre la afirmación	Aprobar un gate con evidencia en prosa; citar una línea sin abrirla
2	Nadie certifica su propio trabajo	Que el verificador reciba el razonamiento del autor
3	Lo que bloquea tiene que poder comprobarse	Afirmaciones bloqueantes de tipo <code>no_verificable</code>
4	Producción se protege sobre la velocidad de entrega	Instalar sin dictamen, sin definición SRE o con hash distinto
5	El alcance no crece en silencio	Completar un diseño ambiguo desde la construcción
6	Los límites van en la infraestructura, no en el texto	Confiar el aislamiento entre roles a la redacción del prompt
7	Toda excepción es explícita, motivada y caduca	Excepciones sin fecha de vencimiento

Cada artículo tiene cuatro partes: principio, por qué (el fallo concreto que evita), prohíbe (para que no haya interpretación) y gates (qué comprueba el cumplimiento).

Un artículo sin gate es decoración

Es la diferencia deliberada frente a otros enfoques de «constitución de proyecto». Aquí un script lo comprueba:

```
./scripts/verificar-constitucion.sh
```

Falla si un artículo no nombra gates, si nombra uno inexistente, o si le falta alguna sección obligatoria.

Lo que encontré la primera vez que se ejecutó

El artículo 7 citaba un gate llamado G7_cierre_completo. La skill del líder técnico lo mencionaba desde la primera versión del framework — pero nunca existió en gates.yaml. Estuvo dos versiones sin que nadie lo notara, precisamente porque nada lo comprobaba. Ese es el argumento entero de la sección, en un caso real.

Por qué cada artículo lleva su razón escrita

Un límite sin razón se cruza en cuanto estorba, y siempre estorba en el peor momento. El artículo 4 es el ejemplo: el coste de un despliegue tardío es visible y acotado; el de uno que rompe algo estable es difuso, tarda en aparecer y lo paga gente que no participó en la decisión. Esa asimetría no se percibe al decidir. Por eso la regla tiene que estar escrita antes.

Los artículos 1, 2, 3 y 6 son el núcleo. Un equipo que necesite retirar alguno probablemente necesita otro framework, no este con menos artículos. El comando `/constitucion` avisa explícitamente cuando un cambio propuesto debilita uno de ellos.

5 · Por qué sirve para reusar

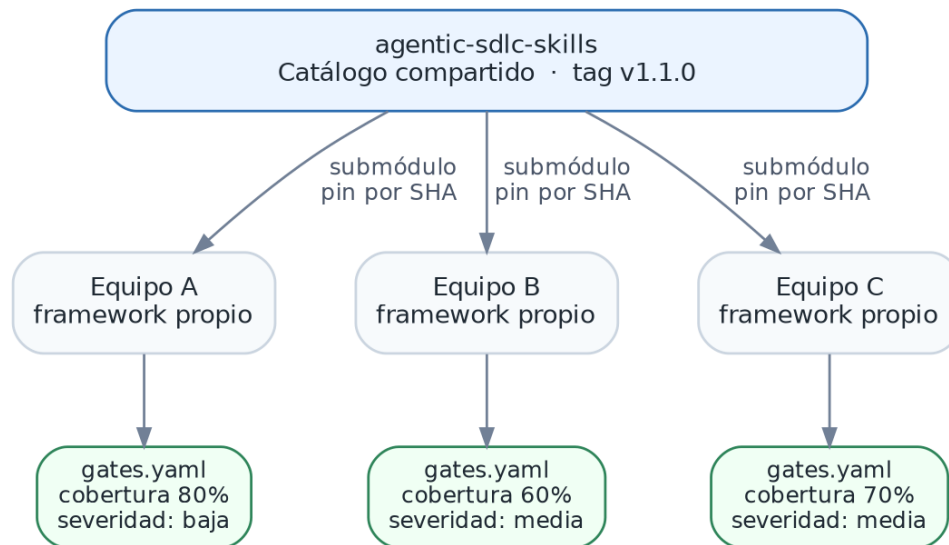


Figura 3 — Un catálogo, muchos equipos. Las mismas instrucciones, distintos umbrales. El pin por SHA del submódulo significa que ningún equipo recibe un cambio de procedimiento sin un commit propio.

Tres equipos con perfiles de riesgo distintos —uno de pagos, uno de herramientas internas, uno de datos— comparten los ocho procedimientos sin discutirlos. Lo que negocian es su `gates.yaml`: el de pagos exige cobertura alta y tolera solo hallazgos bajos; el de herramientas internas es más laxo. Mismas instrucciones, distinta exigencia.

La alternativa —copiar las skills al repo de cada equipo— se descarta porque las copias divergen en semanas y después nadie sabe cuál es la buena. Con el submódulo, la actualización es explícita: un commit que dice «subimos el catálogo de v1.1.0 a v1.2.0», y ahí queda registrado cuándo cambió la forma de trabajar del equipo.

6 · Verificación adversarial

Es el mecanismo que más precisión aporta, y no es un control de seguridad: es un control de exactitud. Un agente que produce un artefacto y luego lo revisa tiene el mismo punto ciego en ambos pasos.

El obstáculo a resolver primero: no se puede verificar prosa. Un revisor enfrentado a párrafos solo puede opinar. Por eso cada rol emite, además de su artefacto, un archivo de afirmaciones: conclusiones discretas, cada una con dónde comprobarla.

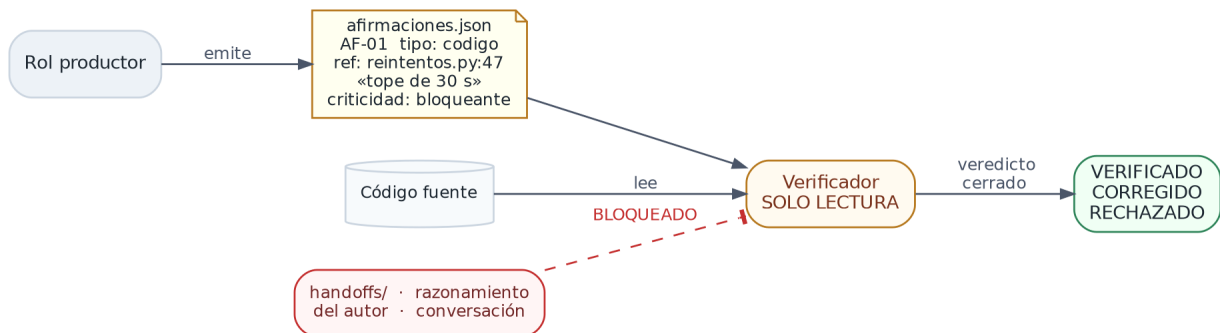


Figura 4 — Ciclo de verificación. El verificador recibe las afirmaciones y el código. Nunca el razonamiento del autor.

```
{ "id": "AF-01",
  "tipo": "codigo",
  "referencia": "src/consumidor/reintentos.py:47",
  "texto": "El backoff exponencial tiene tope de 30 segundos",
  "criticidad": "bloqueante" }
```

El verificador abre ese archivo, y encuentra que la línea 52 aplica jitter hasta 1.5x. Veredicto: CORREGIDO – máximo efectivo 45 s. Es el tipo de imprecisión que un agente autoevaluándose nunca encuentra, porque ya «sabe» lo que escribió.

Los tres invariantes

- Autor y verificador son agentes distintos, y el verificador no ve el razonamiento del autor. Solo las afirmaciones y el código.
- La carga de la prueba se invierte. La instrucción no es «revisa si está bien», es «desmiente esto; si no puedes, confírmalo con la evidencia que te lo impidió».
- El veredicto es cerrado: VERIFICADO, CORREGIDO o RECHAZADO. «Parece correcto pero» no sirve para decidir.

El aislamiento no es negociable

El verificador es el mismo modelo con el mismo entrenamiento. Si le entregas el razonamiento del autor, no verifica: lo reformula. El framework gana una firma de aprobación sin ganar precisión, lo cual es peor que no verificar — produce confianza infundada en algo que nadie comprobó. Por eso el subagente verificador tiene solo herramientas de lectura y su prompt le prohíbe leer los handoffs.

Lo que este mecanismo NO resuelve

Verificador y autor comparten modelo. El aislamiento de contexto mitiga el sesgo compartido; no lo elimina. Si ambos recuerdan mal la misma API, los dos se equivocan igual. Un lote donde todo sale VERIFICADO no significa que esté todo bien: significa que las referencias resuelven y dicen lo que se afirmó. Eso es bastante más de lo que había antes, y bastante menos que una garantía. El gate no sustituye la revisión humana: la enfoca.

7 · Cómo se modifica el framework

Esta sección es la que hay que leer para adoptarlo. La pregunta siempre es la misma: ¿qué archivo toco para conseguir este cambio?

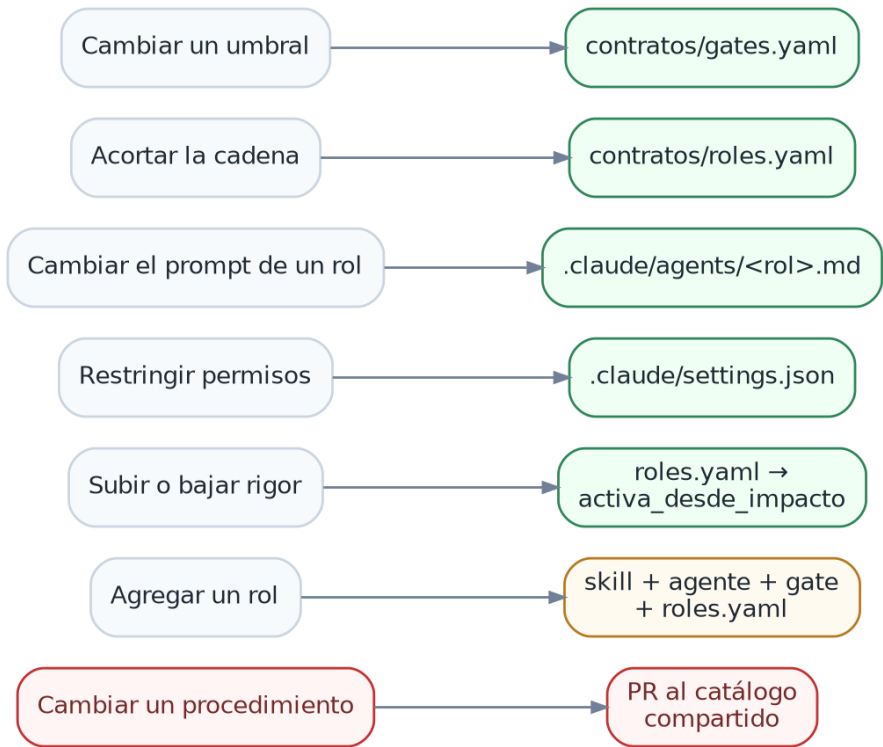


Figura 5 — Mapa de personalización. Verde: cambios rutinarios en tu propio repo. Ámbar: cambio estructural. Rojo: afecta a todos los equipos y va por PR.

7.1 · Los cambios rutinarios

Cinco cosas cubren el 90% de lo que un equipo necesita ajustar, y todas están en su propio repo.

Quiero...	Toco	Qué cambio ahí
Exigir más o menos cobertura	<code>contratos/gates.yaml</code>	<code>G4b_cobertura_minima → umbral</code>
Tolerar hallazgos de otra severidad	<code>contratos/gates.yaml</code>	<code>G5b → severidad_maxima</code>
Que un gate solo aplique a cambios grandes	<code>contratos/gates.yaml</code>	<code>activo_si: ninguno indirecto directo</code>
Permitir saltarse un gate con motivo	<code>contratos/gates.yaml</code>	<code>anulable: true</code>
Acortar la cadena para correcciones	<code>contratos/roles.yaml</code>	<code>tipos.correccion.cadena</code>
Subir o bajar el rigor global	<code>contratos/roles.yaml</code>	<code>verificacion_adversarial.activa_desde_impacto</code>
Restringir qué puede ejecutar un agente	<code>.claude/settings.json</code>	<code>permissions.deny / permissions.ask</code>

Quiero...	Toco	Qué cambio ahí
Cambiar el prompt de un rol	.claude/agents/<rol>.md	cuerpo del archivo (ver 7.2)

La perilla principal

Si solo se toca una cosa, que sea esta:

```
# contratos/roles.yaml
verificacion_adversarial:
  activa_desde_impacto: indirecto # ninguno | indirecto | directo
```

En ninguno se verifica todo y el ciclo cuesta aproximadamente el doble. En directo solo se verifica lo que toca producción. Es el control de rigor contra velocidad, y conviene empezar en indirecto y moverlo con datos, no con intuición.

7.2 · Dónde viven los prompts

Un rol se comporta según tres textos que el agente lee en momentos distintos. Saber cuál toca cambiar evita el error más común al adoptar el framework: engordar el prompt con instrucciones que pertenecen a otro sitio.

Nivel	Archivo	Cuándo se lee	Qué contiene
Contrato del repo	CLAUDE.md	Siempre, en toda sesión	Las reglas que valen para todos los roles: no trabajar sin orden, no dar contexto al verificador, no poner credenciales en un prompt
Prompt del rol	.claude/agents/<rol>.md	Al invocar ese subagente	Identidad en una frase, herramientas permitidas, la frontera con los roles vecinos, y la orden de cargar su skill
Procedimiento	<rol>/SKILL.md	Cuando el rol carga la skill	Entrada, salida, pasos, reglas, gates. El grueso del contenido

Por qué los prompts son tan cortos

Los ocho prompts del framework suman 118 líneas entre todos: entre 12 y 23 cada uno. No es minimalismo por gusto. Un prompt de rol responde solo tres preguntas —quién eres, qué NO haces, qué skill cargas— y todo lo demás vive en la skill, donde puede compartirse entre equipos y versionarse aparte.

El prompt completo del rol SRE, íntegro:

```
---
name: sre
description: Define objetivos de servicio, alertas, runbook y criterio
  de reversión antes del despliegue. Invócalo tras un dictamen favorable.
tools: Read, Grep, Glob, Write, Bash, Skill
---

Eres el rol SRE. Carga la skill `sre` y síguela.

Respondes: cuando esto falle en producción, ¿cómo nos enteramos y qué
hacemos?

Cada alerta apunta a un runbook. El criterio de reversión es numérico,
con ventana. La reversibilidad se prueba en un ambiente no productivo
y se registra: un procedimiento escrito sin ejecutar no es evidencia.

Los umbrales que fijas se escriben en `contratos/gates.yaml`.
```

La línea `tools:` no es decorativa: define qué herramientas tiene el subagente. El verificador, por ejemplo, no lleva `Write` sobre el código — solo lectura y su carpeta de veredictos. Esa línea es un control real, no una sugerencia.

Qué SÍ va en un prompt de rol

- La identidad y la frontera: «no diseñas, no despliegas».
- Las herramientas permitidas, en `tools:`.

- Las dos o tres reglas cuya violación rompería el rol entero — por ejemplo, que el verificador no lea los handoffs.
- La orden de cargar la skill.

Qué NO va en un prompt de rol

- El procedimiento. Va en la skill, que se comparte entre equipos. Un paso metido en el prompt se pierde para los demás equipos y para el versionado del catálogo.
- Umbrales. Van en `gates.yaml`. Sigue siendo la regla número uno, y en un prompt es más tentador que nunca.
- Contexto del proyecto. Va en `CLAUDE.md`; si está en cada prompt, hay ocho copias que se desincronizan.

La prueba antes de tocar un prompt

Si lo que quieres agregar valdría para el mismo rol en otro equipo, va en la skill. Si es un número, va en `gates.yaml`. Si vale para todos los roles, va en `CLAUDE.md`. Solo lo que queda después de esos tres descartes pertenece al prompt del rol — y suele ser muy poco.

7.3 · Agregar un rol

Cinco pasos. Si el rol nuevo no tiene al menos un gate verificable, probablemente no es un rol: es una etapa de otro rol.

- Skill en el catálogo, siguiendo `AUTHORING.md`.
- Subagente en `.claude/agents/` que cargue esa skill y declare sus herramientas.
- Gates en `gates.yaml`, con `rol`: apuntando al nombre del subagente.
- Insertar el rol en la cadena correspondiente en `roles.yaml`.
- Actualizar el diagrama de la documentación.

7.4 · Agregar un gate

Un gate necesita las cuatro cosas o no es un gate:

Elemento	Qué es
Nombre	Estable, referenciado desde el <code>SKILL.md</code> del rol
Verificación	<code>presencia_archivo</code> · <code>comparacion_hash</code> · <code>umbral_numerico</code> · <code>manual</code>
Evidencia	Qué archivo lo demuestra
Condición de aborto	Qué pasa si falla

Un gate `manual` es legítimo, pero cada uno es un punto donde el framework confía en una afirmación en vez de comprobarla. Manténlos en minoría.

7.5 · Lo que NO se toca

Los `SKILL.md` del catálogo compartido. Si aparece la necesidad de editar uno, casi siempre es una de dos cosas:

- Querías cambiar un umbral. Va en `gates.yaml`; el catálogo ya lo contempla.
- Tu equipo tiene un paso que otros no. Proponlo al catálogo como PR, o agrega una skill local en `.claude/skills-locales/` y referénciala desde el subagente.

No edites el submódulo directamente

Un cambio hecho dentro de `.claude/skills/` desaparece en el próximo git submodule update, normalmente sin aviso y en el peor momento. Si el cambio vale para todos, va por PR al catálogo; si es solo tuyo, va en una skill local.

8 · Portabilidad entre plataformas

El mismo repositorio funciona en Claude Code, en Codex y en cualquier agente que lea AGENTS.md. No hay dos versiones que mantener: hay una fuente y un generador.

El problema es que cada plataforma lee un archivo distinto. AGENTS.md se ha vuelto el estándar de facto —está bajo la Agentic AI Foundation de la Linux Foundation y lo leen Codex, Cursor, Windsurf y Cline— pero Claude Code sigue leyendo CLAUDE.md. Mantener ambos a mano garantiza que diverjan, y el día que alguien cambie un límite en uno y no en el otro, dos agentes del mismo equipo estarán operando bajo reglas distintas sin que nadie lo sepa.

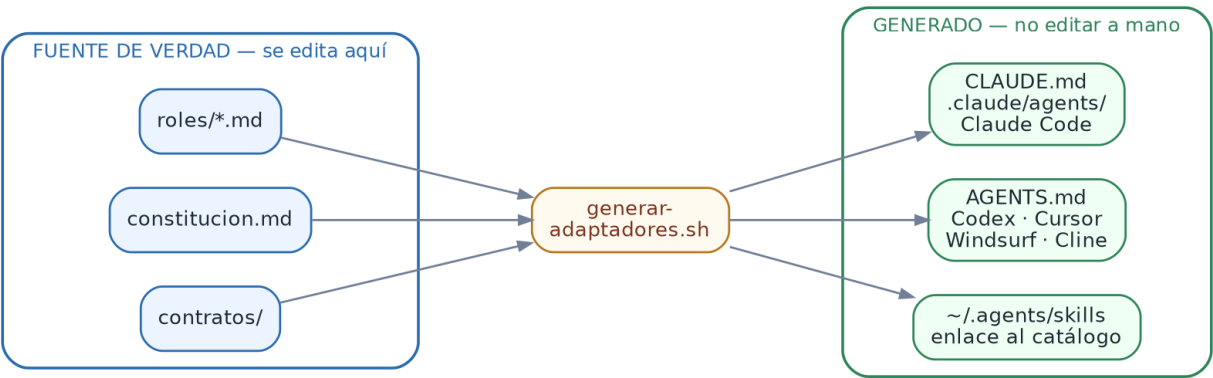


Figura 6 — Una fuente, varios adaptadores. Los archivos generados llevan marca de generado en la primera línea; editarlos a mano es un cambio que el siguiente bootstrap borra.

	Claude Code	Codex · Cursor · Windsurf · Cline
Instrucciones	CLAUDE.md	AGENTS.md
Roles	Subagentes en .claude/agents/	Descritos en AGENTS.md, se adoptan uno a uno
Comandos	/iniciar · /verificar · /gate	Se invocan los scripts directamente
Skills	.claude/skills/	Enlace en ~/.agents/skills
Aislamiento del verificador	Real: subagente de solo lectura	Parcial: exige abrir sesión nueva

La diferencia que importa de verdad

En plataformas sin subagentes, el aislamiento del verificador no se puede simular con buena voluntad. Si el mismo hilo produce las afirmaciones y luego las verifica, el verificador ya vio el razonamiento del autor y va a reformularlo, no a comprobarlo. Por eso AGENTS.md instruye abrir una sesión nueva para verificar: es menos cómodo y funciona.

9 · Instalación paso a paso

Requisitos: git, python3 (3.8+) y bash. Sin Node, sin Docker, sin servicios que levantar, sin paquetes que instalar. Diez minutos la primera vez, dos las siguientes.

9.0 · Publicar los dos repos (una sola vez)

```
# 1. El catálogo – compartido entre todos los equipos
cd agentic-sdlc-skills
git init && git add -A && git commit -m 'Catálogo v1.2.0'
git tag v1.2.0
git remote add origin
git push -u origin main --tags

# 2. El framework – cada equipo tendrá el suyo
cd ../agentic-sdlc-framework
git init && git add -A && git commit -m 'Framework v1.2.0'
git submodule add .claude/skills
cd .claude/skills && git checkout v1.2.0 && cd ../../
git add .gitmodules .claude/skills && git commit -m 'Pin v1.2.0'
git remote add origin && git push -u origin main
```

Nada depende de un proveedor Git concreto: si no usas GitHub, cambia la URL.

9.1 · Camino A — Coder

El caso del equipo destinatario: contenedor con salida solo al servidor Git.

```
# En el main.tf del template, copiar el bloque de coder/main.tf y ajustar:
variable "framework_repo" { default = "" }
variable "framework_ref" { default = "v1.2.0" } # pinear, nunca main

coder templates push
```

Al arrancar el workspace, el script clona con submódulos y corre bootstrap.sh solo. No hay paso manual. Comprobar con ./scripts/prueba-humo.sh.

Antes de llevarlo a producción

Crea un template aparte para el rol de despliegue, con las credenciales productivas y una allowlist de red más estrecha. El template general no debe tenerlas nunca — es el artículo 6 de la constitución aplicado a la infraestructura.

9.2 · Camino B — Claude Code

```
git clone --recurse-submodules
cd agentic-sdlc-framework
./scripts/bootstrap.sh
claude
```

Claude Code lee CLAUDE.md al arrancar y descubre solo los subagentes y los comandos. Comprueba con /constitucion: debe listar los siete artículos y confirmar que todos tienen gate.

Si ya lo habías clonado sin submódulos: `git submodule update --init --recursive`.

9.3 · Camino C — Codex

```
git clone --recurse-submodules
cd agentic-sdlc-framework && ./scripts/bootstrap.sh

# si tu Codex busca skills en otra ruta:
mkdir -p ~/.agents && ln -sf "$(pwd)/.claude/skills" ~/.agents/skills

codex
```

Codex lee AGENTS.md, generado de la misma fuente que CLAUDE.md. Como no tiene subagentes, los roles se adoptan uno a uno:

```
Actúa como lider-tecnico según AGENTS.md.
Solicitud: agregar reintentos con backoff al consumidor de eventos.
```

Los comandos slash no existen aquí; se invocan los scripts directamente (./scripts/verificar-gates.sh, ./scripts/validar-salida.sh).

La única parte que no se puede improvisar

Para verificar, abre una sesión nueva. Si verificas en el mismo hilo, el verificador ya vio el razonamiento del autor y va a reformularlo en vez de comprobarlo. Es incómodo y es lo que hace que el veredicto valga algo.

9.4 · Camino D — Cursor, Windsurf, Cline

Clonar y correr bootstrap.sh igual que en B. Los tres auto-descubren AGENTS.md en la raíz y lo cargan en su motor de reglas: no hay configuración adicional. A partir de ahí, igual que Codex — roles uno a uno y sesión nueva para verificar.

9.5 · Paso final, común a todos

```
$EDITOR contratos/gates.yaml    # los dos umbrales marcados AJUSTAR
$EDITOR contratos/roles.yaml    # activa_desde_impacto

./scripts/verificar-constitucion.sh
./scripts/prueba-humo.sh        # -> 15 correcto(s), 0 fallo(s)

git add -A && git commit -m 'Umbrales del equipo'
```

Solución de problemas

Síntoma	Causa	Salida
catálogo de skills no montado	Clonado sin --recurse-submodules	git submodule update --init --recursive
CONSTITUCIÓN NO EXIGIBLE	Un artículo cita un gate inexistente	Agregar el gate o corregir la cita
el submódulo NO está en el SHA pinneado	Alguien lo movió a mano	git submodule update --init
El agente ignora los roles	Editaste CLAUDE.md o AGENTS.md a mano	Editar roles/ y regenerar adaptadores
bloqueante no_verificable rechazada	Afirmación sin referencia comprobable	Añadir ruta:linea o bajar a informativa

10 · Adopción organizativa

Requisitos: un contenedor con un agente CLI que lea `.claude/`, `git`, `python3` y salida de red hacia el servidor Git. Nada más — sin servicios, sin registro de paquetes, sin dependencias que instalar.

#	Paso	Resultado
1	Publicar los dos repos: catálogo y framework	El catálogo se comparte; el framework se forkea por equipo
2	Enganchar el submódulo y pinearlo a un tag	El equipo controla cuándo cambia su procedimiento
3	Ajustar <code>gates.yaml</code> y <code>roles.yaml</code>	Los umbrales pasan a ser los del equipo
4	Separar el entorno con credenciales productivas	Solo el rol de despliegue las tiene
5	Correr <code>./scripts/prueba-humo.sh</code>	15 comprobaciones: contratos, constitución, validadores, adaptadores
6	Correr una orden de punta a punta en un cambio real y pequeño	El equipo ve la cadena completa antes de confiar en ella

El aislamiento va en la infraestructura, no en el prompt

El rol de despliegue debe correr en un entorno distinto, con las credenciales productivas y una allowlist de red más estrecha. Un prompt puede decir «no toques producción» y el modelo puede cumplirlo casi siempre. Casi siempre no es una garantía. Si el contenedor no tiene la credencial, no hay nada que cumplir.

Primera corrida

```
git clone --recurse-submodules
cd agentic-sdlc-framework && ./scripts/bootstrap.sh

/iniciar   Agregar reintentos al consumidor de eventos
/verificar      # verificación adversarial del rol que terminó
/gate          # qué falta para que la cadena avance
/constitucion   # los límites, y si siguen siendo exigibles
```

Elige para la primera corrida un cambio real pero pequeño, con impacto declarado indirecto. Un cambio grande hace que el equipo culpe al framework de la fricción propia del cambio; uno trivial no activa suficientes gates como para que se vea el valor.

11 · Fuentes y créditos

Este framework no se inventó de cero. Se ensambló a partir de trabajo público de otros, y esta sección dice qué se tomó de dónde, bajo qué licencia, y —tan importante como lo anterior— qué se evaluó y se descartó, con la razón. Eso último evita que el siguiente equipo repita la evaluación.

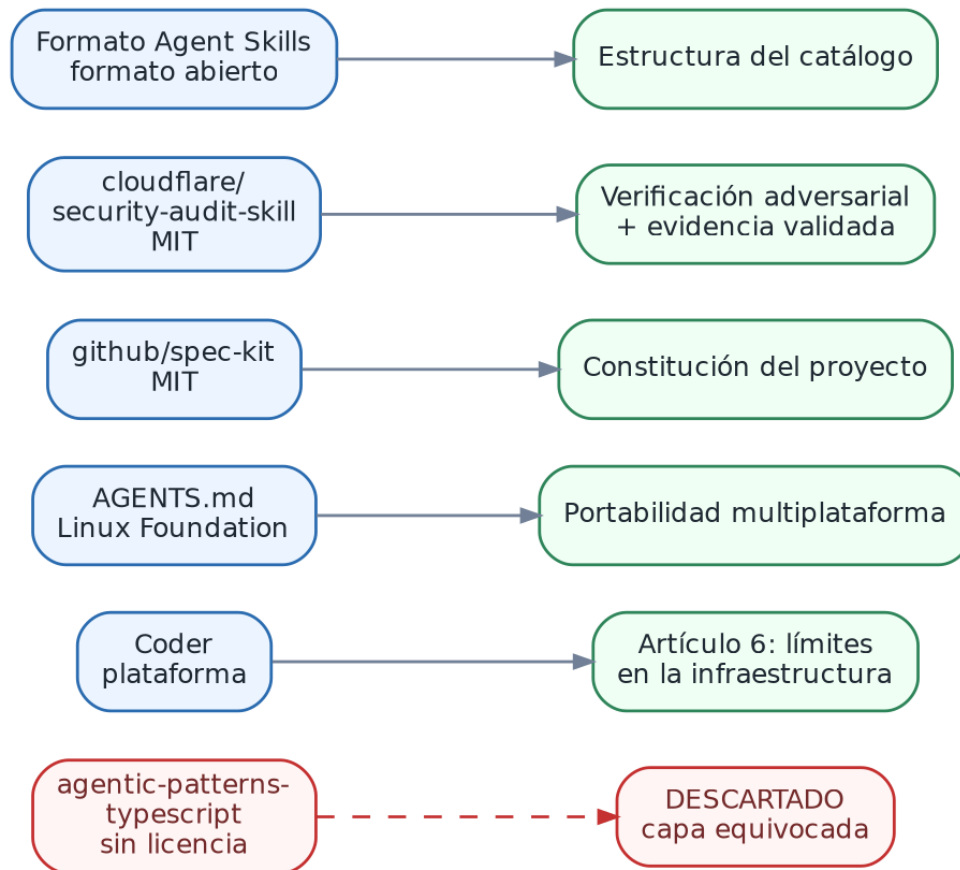


Figura 7 — De cada fuente, lo que se tomó. En rojo, lo evaluado y descartado.

Lo que se adoptó

Fuente	Licencia	Qué se tomó
Formato Agent Skills	Formato abierto	La estructura del catálogo. Es la razón de que no quede atado a una plataforma.
cloudflare/security-audit-skill	MIT	El rol adversarial separado, la evidencia validada por esquema, y los criterios de severidad con impacto obligatorio.
github/spec-kit	MIT	El concepto de constitución del proyecto.
AGENTS.md (Linux Foundation)	Convención abierta	El formato como destino de generación multiplataforma.
Coder	Producto comercial	El modelo de template declarativo, y el argumento de que los controles de seguridad no viven en el prompt — que es el artículo 6.

Dónde nos separamos de las fuentes

Frente a Cloudflare. Su patrón adversarial funciona porque cada hallazgo de seguridad trae `file:line`. Generalizarlo a todos los roles obligó a inventar el archivo de afirmaciones, porque no se puede verificar prosa. Ese fue el trabajo propio.

Frente a spec-kit. Allí la constitución son principios en prosa. Aquí cada artículo debe nombrar los gates que lo comprueban y un script lo verifica. Además, spec-kit es horizontal —las fases de construir una feature— y esto es vertical —quién tiene autoridad sobre qué—. No son alternativas: spec-kit puede vivir dentro de los roles de arquitecto y constructor.

Lo que se evaluó y se descartó

betta-tech/agentic-patterns-typescript — ejemplos de patrones agénticos (pipeline, router, reflexión, orquestador-trabajadores) en TypeScript.

- Capa equivocada. Construye agentes en código, llamando a una API desde Node. Este framework no ejecuta agentes: los ejecuta el CLI del workspace. Adoptarlo significaría un servicio que mantener.
- Red. Necesita instalar paquetes; el contenedor destino solo alcanza el servidor Git.
- Sin licencia declarada en el repositorio, lo que por sí solo impide adoptarlo en un entorno corporativo.

Sí aportó una idea: su patrón router es conceptualmente el mismo que el líder técnico —clasificar primero, activar un único especialista después, derivar a humano con baja confianza—. Eso reforzó la regla de bloqueo hacia el líder técnico. Ideas de diseño, no código.

Lo que es original de este framework

- La separación en siete roles con contratos de handoff y frontera explícita.
- Los gates como dato, con `activo_si` y `anulable`.
- El archivo de afirmaciones verificables, que hace posible generalizar la verificación adversarial fuera del dominio de seguridad.
- La regla de que cada artículo de la constitución nombre sus gates, con script que lo comprueba.
- La capa de portabilidad: fuente neutra más generador de adaptadores.
- El estándar de autoría de skills.

Ninguna de las fuentes MIT se redistribuye aquí: se tomaron patrones e ideas, reescritos. Si en el futuro se incorpora código de alguna de ellas, hay que incluir su aviso de copyright.

12 · Costo, caché y medición

Una orden completa son siete invocaciones de rol más las pasadas del verificador: alrededor de trece llamadas donde antes había una. Cada una vuelve a pagar el contexto fijo. Frente a un agente suelto haciendo la misma tarea, la relación realista está entre 3x y 6x, no en el doble.

Eso es corregible sin tocar la arquitectura, y la palanca es grande.

12.1 · La frontera de caché

El proveedor reutiliza el cómputo del prefijo idéntico de una petición. La medición pública de referencia sobre más de 500 sesiones agénticas: cachear reduce el costo de API entre 41% y 80% y mejora el time-to-first-token entre 13% y 31%. El matiz importa: cachear solo el contexto estable rinde más que cachear todo, que puede incluso aumentar la latencia.

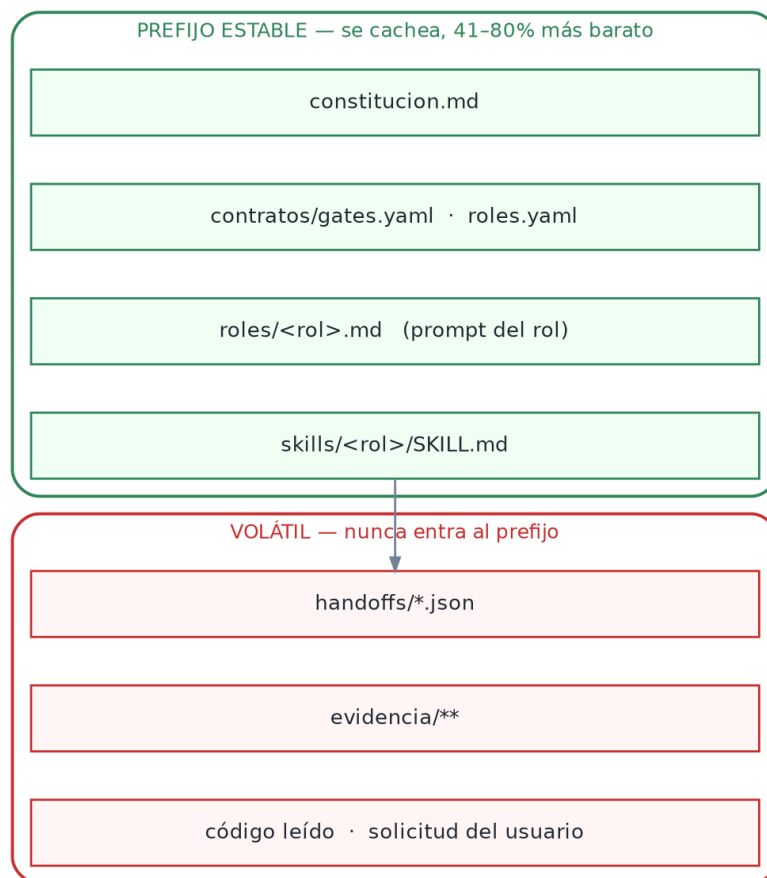


Figura 8 — Frontera de caché. Estable arriba, volátil abajo. El framework ya tenía esta separación por otras razones; declararla la vuelve accionable.

La separación no hubo que inventarla: constitución, gates, prompt de rol y skill son estables por diseño; handoffs, evidencia y código son volátiles. Está declarada en contratos/contexto.yaml, con dos consecuencias operativas: hay un mínimo de tokens por debajo del cual la caché no compensa, y una ventana de reutilización — partir una orden en varios días paga el prefijo cada vez.

12.2 · Mover del agente al script

Este es el ahorro propio del framework, y es del 100% en lo que cubre, no del 80%. Antes de invocar al verificador, `resolver-referencias.sh` comprueba sin gastar un token que cada referencia resuelve —el archivo existe, la línea está en rango, el ejecutable está disponible— y extrae el contenido citado.

Dos efectos. Una referencia rota ya es CORREGIDO o RECHAZADO sin invocar a nadie. Y cuando todas resuelven, el verificador recibe las líneas ya extraídas: le queda el juicio, no la búsqueda.

La regla general

Toda comprobación que un script pueda hacer es una que el agente no debe hacer. Los cuatro gates marcados manual que quedan son la lista de trabajo pendiente en esta dirección: cada uno es un agente haciendo aritmética que sale gratis en bash.

12.3 · Medir en vez de opinar

La pregunta «¿es eficiente?» no tiene respuesta hoy en ningún equipo que no instrumente. Es exactamente el vacío que motivó a la Linux Foundation a anunciar la Tokenomics Foundation en junio de 2026, en alianza con la FinOps Foundation: no existía un lugar neutral para definir cómo se mide la eficiencia de tokens, y el trabajo incluye extender la especificación FOCUS de FinOps al gasto basado en tokens.

Mientras esos estándares maduran, el framework registra lo mínimo indispensable:

```
./scripts/registrar-costos.sh
./scripts/reporte-costos.sh # o /costo
```

El reporte da dos números que hay que leer juntos:

Número	Qué significa	Qué hacer si sale mal
% de prefijo reutilizado	Cuánto está funcionando la caché	Por debajo del 50%, hay contenido volátil colado en el prefijo estable
Tasa de hallazgo de la verificación	Qué proporción de afirmaciones resultó tener algo mal	Si es cero tras varias órdenes, lo más probable es que el verificador esté recibiendo contexto del autor y solo confirme

Lo que este documento NO afirma

No hay medición propia de que el framework mejore la precisión. El argumento es de diseño —separar quien afirma de quien comprueba— y hay una anécdota a favor: el verificador de la constitución encontró un gate roto que llevaba dos versiones sin detectarse. Una anécdota no es una tasa. La tasa de hallazgo del reporte es precisamente el número que convierte esa creencia en dato, y hay que dejar correr diez órdenes reales antes de defender nada.

12.4 · Dónde no usarlo

Un cambio trivial, un prototipo, una exploración. Para eso existe `activa_desde_impacto`. Un equipo que lo deje en ninguno y lo aplique a todo va a odiarlo en dos semanas, con razón.

Y la comparación justa no es «framework contra agente suelto», sino «framework contra agente suelto más el tiempo humano de revisar lo que produjo». Si hoy nadie revisa nada, el framework es puro costo añadido. Si alguien revisa, ahí empieza a pagarse.

13 · Ingeniería de contexto

El eje que faltaba. Las guías anteriores cubren cómo se estructura una skill y qué sale de ella. Esta cubre qué entra antes de que corra — porque una skill impecable con el contexto equivocado produce una respuesta impecable y equivocada, que es peor que una mala: nadie la revisa.

El cambio de pregunta: la ingeniería de prompt pregunta ¿cómo lo escribo? y si falla, reformulas. La ingeniería de contexto pregunta ¿qué necesita saber el modelo ahora? y si falla, revisas de dónde salió la información. Uno es redacción; el otro, arquitectura.

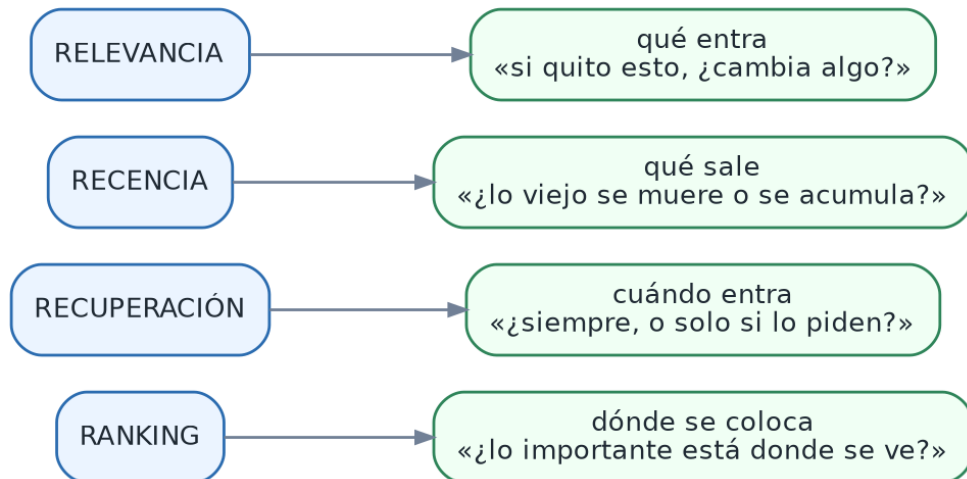


Figura 9 — Las cuatro decisiones y su pregunta de auditoría. Aplicar solo una arreglo la cuarta parte del problema.

13.1 · La tensión que había que resolver

Dos optimizaciones que tirábamos en direcciones opuestas sin darnos cuenta:

- La caché exige que lo estable vaya primero: el proveedor reutiliza el cómputo del prefijo idéntico.
- La atención exige que lo importante vaya al principio o al final. La misma información rinde 70-75% de exactitud en los extremos y 55-60% en la mitad — quince a veinte puntos solo por la posición.

Puestas juntas mal, el prefijo estable empuja la tarea real hacia la mitad —la peor posición— y se gana en costo lo que se pierde en exactitud. La versión anterior de este framework tenía exactamente ese defecto.

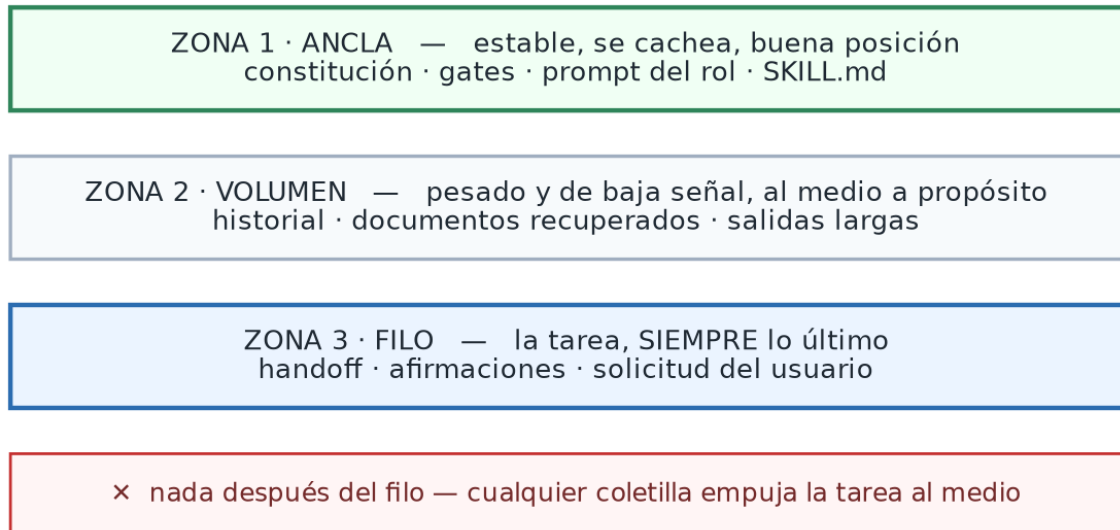


Figura 10 — Tres zonas en vez de dos. El ancla ocupa el inicio (cacheada y bien posicionada), la tarea el final, y el volumen de baja señal el medio, donde menos daño hace.

La regla que hace que funcione

Nunca escribas nada después de la zona filo. Ni un recordatorio, ni un «responde en formato X», ni una repetición de las reglas. Cualquier coletilla empuja la tarea al medio y la degrada. Si una restricción es crítica y hay que repetirla, va en la zona ancla —donde ya está cacheada y bien posicionada—, no como cierre.

13.2 · Presupuesto de contexto

Lo que no se mide, crece. El mapa por rol:

```
./scripts/mapa-contexto.sh # o /contexto
```

La primera corrida sobre este mismo framework dio un resultado útil: la zona ancla pesa unas 1.800 palabras, de las cuales el 80% es constitución más gates y el prompt del rol apenas el 8%. La parte compartida se cachea una vez, así que el diseño es correcto — pero deja claro que si algo hay que podar, es la constitución, no los prompts.

Señal	Qué significa
El historial supera el 60%	Nadie lo administra; falta recencia
Más de 20 herramientas declaradas	Un benchmark citado: falló con 46, funcionó con 19
La zona ancla crece versión tras versión	Nadie ha corrido la pregunta de relevancia
Documentos precargados «por si acaso»	Candidatos a recuperación bajo demanda

Un detalle que sorprende: las definiciones de herramientas ocupan contexto aunque no se usen. Un rol con cuarenta declaradas y seis en uso paga las cuarenta en cada mensaje.

13.3 · La crítica que esto le hace al framework

La pregunta de relevancia —si quito esta línea, ¿cambia alguna respuesta?— no se ha corrido nunca sobre este documento ni sobre la constitución. La tendencia natural de todo framework es acumular instrucciones, y la evidencia apunta en contra: Anthropic recortó más del 80% del system prompt de su herramienta de código para los modelos Claude 5, y el benchmark de herramientas muestra que quitar capacidades puede mejorar el resultado.

Es trabajo pendiente reconocido, no una sugerencia.

Procedencia de las cifras de esta sección

Proviene de material formativo sobre ingeniería de contexto y no han sido verificadas de forma independiente por este framework. Trátalas como órdenes de magnitud que orientan una decisión de diseño, no como constantes. El diseño de tres zonas se sostiene aunque las cifras difieran, porque la dirección del efecto —los extremos rinden más que el medio— está bien establecida.

14 · Límites conocidos

Un framework que solo enumera sus virtudes es difícil de adoptar bien, porque el equipo descubre los límites en producción en vez de en la evaluación. Estos son los que se conocen.

Es secuencial a propósito. No hay ejecución en paralelo. El valor está en la trazabilidad, no en la velocidad. Un equipo que necesite throughput por encima de auditabilidad debería mirar otra cosa.

Verificador y autor comparten modelo. Ya tratado en la sección 5. El aislamiento de contexto mitiga el sesgo compartido; no lo elimina.

Los gates manuales son un acto de fe. Cuatro de los gates de referencia se marcan manual: el framework confía en que el agente los evaluó. Convertirlos en verificables por script es la vía de mejora más rentable que queda abierta.

No impide trabajar por fuera. Cualquiera puede hacer el cambio sin pasar por la cadena. Lo que impide es que ese trabajo pase por certificado.

Cuesta más. Entre 3x y 6x frente a un agente suelto, antes de aplicar caché. Ver la sección 12 para las mitigaciones y para cómo medirlo en tu propio equipo en vez de creerse esta cifra.

La precisión no está medida. El argumento a favor es de diseño, no empírico. Es el límite más importante de esta lista.

15 · Resumen de una página

Pregunta	Respuesta
¿Qué hace reutilizable el framework?	Los procedimientos son compartidos; los umbrales son de cada equipo. Ningún número vive dentro de una instrucción.
¿Qué toca un equipo al adoptarlo?	gates.yaml y roles.yaml. Casi siempre nada más.
¿Qué no debe tocar nunca?	Los SKILL.md del catálogo compartido, ni el submódulo directamente.
¿Qué hace que los gates funcionen?	Que son datos leídos por un script, no instrucciones en un prompt.
¿Qué aporta el verificador adversarial?	Precisión. Separa quien afirma de quien comprueba, e invierte la carga de la prueba.
¿Cuánto cuesta de más?	3x-6x frente a un agente suelto, antes de caché. Con la frontera de caché declarada, entre 41% y 80% de eso se recupera.
¿Dónde va cada cosa en la ventana?	Tres zonas: ancla (estable, se cachea), volumen (al medio), filo (la tarea, siempre al final). Nada después del filo.
¿Cuál es la única perilla que hay que entender?	verificacion_adversarial.activa_desde_impacto — rigor contra velocidad.
¿Qué NO se ajusta nunca?	La constitución. Siete artículos, cada uno con los gates que lo comprueban.
¿Funciona fuera de Claude Code?	Sí. Una fuente neutra genera CLAUDE.md y AGENTS.md; el segundo lo leen Codex, Cursor, Windsurf y Cline.
¿Sustituye la revisión humana?	No. La enfoca: muestra qué falta y qué no se pudo comprobar.